



[Hochschule Düsseldorf]
[DAISY - Big Data - SS25]

Image Recommender

Projektbericht

Team: Julia Moor (934797) | Dalia Salih (953818) |

Abgabe: [Abgabedatum: 17.08.2025]

Inhaltsverzeichnis

1	Motivation und Zielsetzung	2
2	Programmdesign und Architektur	2
2.1	Komponenten und Interaktionen	2
2.2	Verwendete Bibliotheken	3
2.3	Datenbankschema	3
3	Schnellstart: CLI & Streamlit	3
3.1	CLI in 60 Sekunden	3
3.2	Streamlit: Interaktive Suche	4
4	Methoden zur Bildähnlichkeit	7
4.1	Farbfeatures (Histogramme, Stats, dominante Farben)	7
4.2	Deep Embeddings (MobileNet V3, 128D)	7
4.3	Custom-Maß: pHash + Varianz + Entropie	7
4.4	Score-Fusion und Re-Ranking	7
4.5	Vergleich: Vor- und Nachteile	8
5	Pipeline: Ingestion und Backfill	8
5.1	Ingestion (schnelles Einlesen)	8
5.2	Embedding-Backfill (batchweise, MPS/GPU)	8
6	Suche und ANN	9
6.1	NumPy-Cosine vs. HNSW	9
7	Performanceanalyse	9
7.1	Performanceanalyse und Optimierung	9
8	Machbarkeitsanalyse	10
9	Big-Data-Analyse: 2D/3D-Visualisierung	11
10	Visualisierung der Embeddings	11
10.1	Overlay: Helligkeit (Brightness)	11
10.2	Overlay: Farbton (Hue)	11
10.3	Einordnung für die Suche	11
11	Tests und Qualitätssicherung	14
11.1	Linting & Formatting (Ruff)	14
11.2	Unit-Tests (pytest)	14
12	Schwierigkeiten und Lessons Learned	14
13	Fazit	15
A	Anhang A: Datenbankschema (SQL)	15

1 Motivation und Zielsetzung

In unserem Projekt haben wir eine skalierbare Software entwickelt, die für ein Eingabebild innerhalb weniger Sekunden die fünf ähnlichsten Bilder aus einem sehr großen Korpus ($\approx 500,000$ Bilder) findet. Die Kernidee ist, mehrere komplementäre Ähnlichkeitsmaße zu kombinieren: (i) farbprofilbasierte Ähnlichkeit (Histogramme und statistische Kennzahlen), (ii) tiefe, komprimierte Bildrepräsentationen (*Deep Embeddings* aus MobileNet V3, auf 128 Dimensionen projiziert) und (iii) ein custom Maß mit *perceptual hashing* (pHash) und Kontrast-/Entropieindikatoren.

Die wesentlichen Ziele:

- **Funktional:** Top-5-Suche pro Query, später auch Multi-Query (2–3 Eingabebilder) und Unterstützung eines ANN-Indexes (HNSW).
- **Performance:** Schneller Ingest (≥ 90 imgs/s auf NVMe/SSD mit JPEG und Downscaling) und schneller Embedding-Backfill (≥ 90 imgs/s).
- **Saubere Architektur:** Relationale DB (SQLite/WAL), klare Schnittstellen, reproduzierbare Pipelines und testbare Similarity-Funktionen.

2 Programmdesign und Architektur

2.1 Komponenten und Interaktionen

Unsere Software ist in klar getrennte Komponenten gegliedert:

1. **Image Ingestion (ImageDatabase.py):** Iteriert über Bilddateien, decodiert (bevorzugt via pyTurboJPEG mit 1/8-Scaling), optionales Resize, extrahiert Farbfeatures (Histogramme in HSV/BGR, dominante Farben in Lab/BGR, Helligkeits-/Varianzstatistiken) und berechnet pHash. Ergebnisse werden in eine SQLite-Datenbank (WAL) geschrieben.
2. **Embedding Backfill (EmbedBackfill.py):** Lädt Bilder in Threads, führt *batchweise* einen Forward-Pass durch MobileNet V3 aus (MPS/GPU, falls verfügbar), projiziert auf 128D und speichert die Embeddings als BLOB.
3. **Recommender (ImageRecommender.py):** Lädt günstige Vektoren (Farb-/Statistikfeatures) und Deep-Embeddings, baut je einen Index (NumPy-Cosine; bei großen Daten HNSW), berechnet die Kandidatenmenge und re-rankt mit einem gewichteten Gesamtscore.
4. **Feature-Module (FeatureExtraction.py, DeepEmbedding.py):** Enthalten die eigentliche Featureberechnung (Farbe, Textur, pHash) und die Deep-Embedding-Pipeline (MobileNet V3).
5. **CLI (cli.py):** Kommandos für *ingest*, *query* (Single) und *query-multi* (Multi-Input), sowie DB-Statistiken.

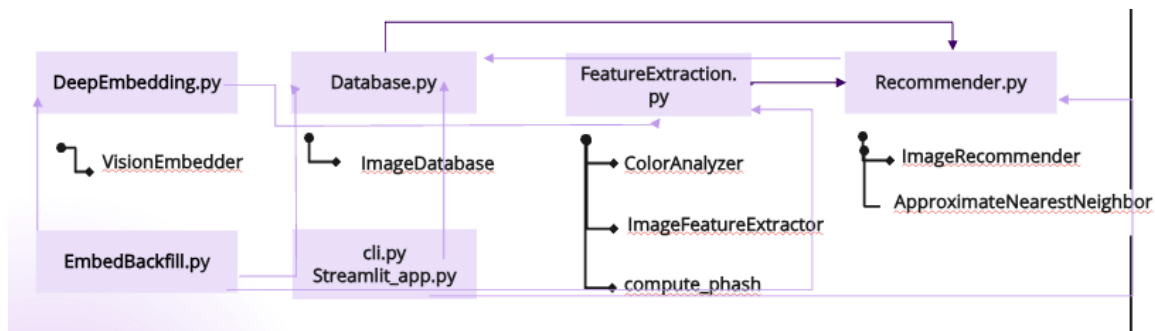


Abbildung 1: Architekturübersicht unserer Software.

2.2 Verwendete Bibliotheken

- **OpenCV** (cv2) für Decoding, Farbräume, Histogramme, Kanten-/Gradientenoperationen.
- **pyTurboJPEG** für extrem schnelles JPEG-Decoding mit *compressed-domain scaling*.
- **NumPy** für Vektoroperationen.
- **PyTorch & TorchVision** für das vortrainierte MobileNet V3 und MPS/GPU-Ausführung.
- **SQLite** (mit **WAL**-Journal) als relationale Datenbank.
- **Optional: hnswlib** für HNSW-Index (Approximate Nearest Neighbor).

2.3 Datenbankschema

Wir verwenden drei zentrale Tabellen:

- `images(image_id TEXT PRIMARY KEY, filename, filepath, file_size, width, height, channels, file_hash, photographer, created_date, added_to_db)`
- `color_features(image_id TEXT PRIMARY KEY, dominant_colors, hsv_histogram, bgr_histogram, color_stats)`
- `advanced_features(image_id TEXT PRIMARY KEY, texture_features, deep_embeddings BLOB, custom_features, deep_dim INT)`

Die `image_id` ist ein stabiler MD5-Hash des *absoluten Pfads* (optional zusätzlich Datei-MD5 für Dedup).

3 Schnellstart: CLI & Streamlit

Dieser Abschnitt beschreibt die schnellste Art, unser System lokal zu benutzen – einmal per **CLI** (Batchprozesse, Skripting) und einmal per **Streamlit**-Weboberfläche (interaktiv).

3.1 CLI in 60 Sekunden

```
1  # 1) Ingestion (Farbhistogramme, Stats, pHash in die DB schreiben)
2  python ImageDatabase.py \
3      --ingest "/pfad/zubildern" \
4      --db image_recommender.db \
5      --pool thread --workers 64 \
6      --decode-mode auto --jpeg-reduce 8 \
7      --bins 16 --k 3 --resize 224 \
8      --batch-size 8000 --log-every 1000
9
10 # 2) Backfill (Deep-Embeddings in die DB schreiben)
11 python EmbedBackfill.py \
12     --db image_recommender.db \
13     --batch 192 --io-workers 16 \
14     --decode-mode auto --jpeg-reduce 8 \
15     --log-every 5
16
17 # 3) Query (Single-Image Top-5)
18 python cli.py query \
19     --db image_recommender.db \
20     --image "/pfad/bild.jpg" \
21     --topk 5 --candidates 200
```

```

22
23 # 3b) Query (Multi-Input, 23 Bilder; Features werden gemittelt)
24 python cli.py query-multi \
25     --db image_recommender.db \
26     --images "/pfad/a.jpg" "/pfad/b.jpg" \
27     --topk 5 --candidates 300

```

Hinweise.

- Bei sehr großen Datenbanken ist der ANN-Index (**hnswlib**) aktiv; für Kaltstarts lohnt sich ein *Index-Cache* (siehe Streamlit).
- `--candidates` kontrolliert die Größe der Kandidatenmenge vor dem finalen Re-Ranking.

3.2 Streamlit: Interaktive Suche

Start der App (im Projektordner):

```

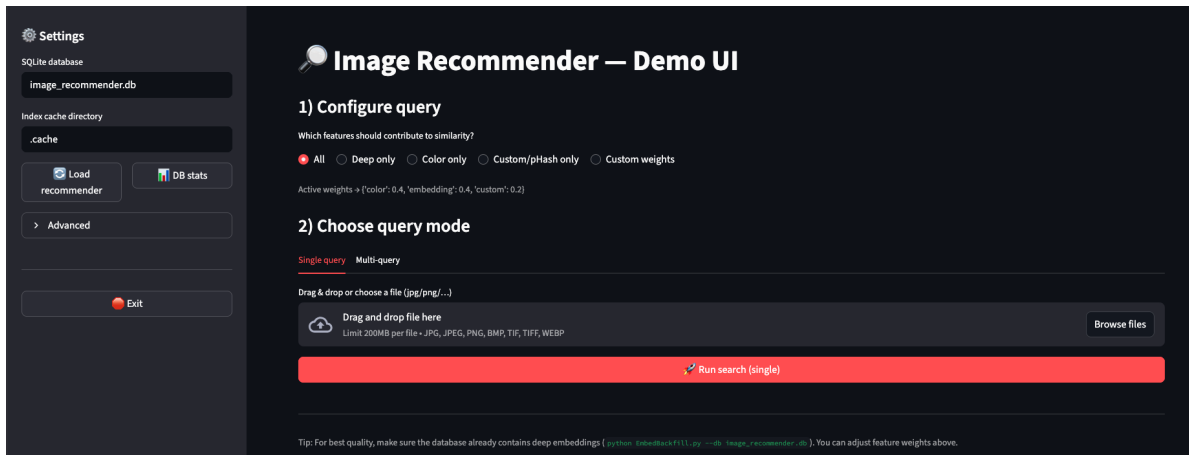
1 streamlit run streamlit_app.py

```

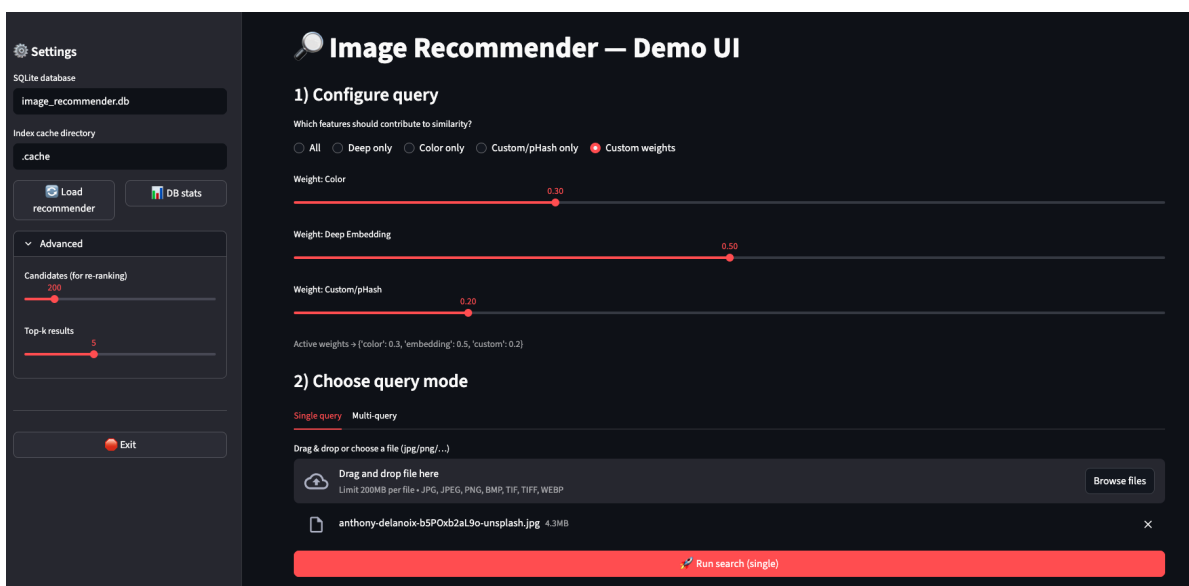
Wichtig ist ein gültiger Pfad zur SQLite-DB (Standard: `image_recommender.db`). Optional kann ein **Index-Cache-Verzeichnis** angegeben werden (z. B. `.cache`); dadurch werden HNSW-Indizes nach dem ersten Build geladen statt neu aufgebaut.

UI-Überblick.

- **Settings (Sidebar):** DB-Pfad, Cache-Verzeichnis, Buttons *Load recommender*, *DB stats*, *Exit*.
- **Weights:** Presets (*All*, *Deep only*, *Color only*, *Custom/pHash only*) oder stufenlos per Slidern (*Custom weights*); Gewichte werden normalisiert.
- **Query-Mode:** *Single query* (ein Bild) oder *Multi-query* (2–3 Bilder; Features/Embeddings werden standardmäßig gemittelt).
- **Advanced:** `candidates` (Größe der Vorselektion) und `top-k` (Anzahl der finalen Treffer).

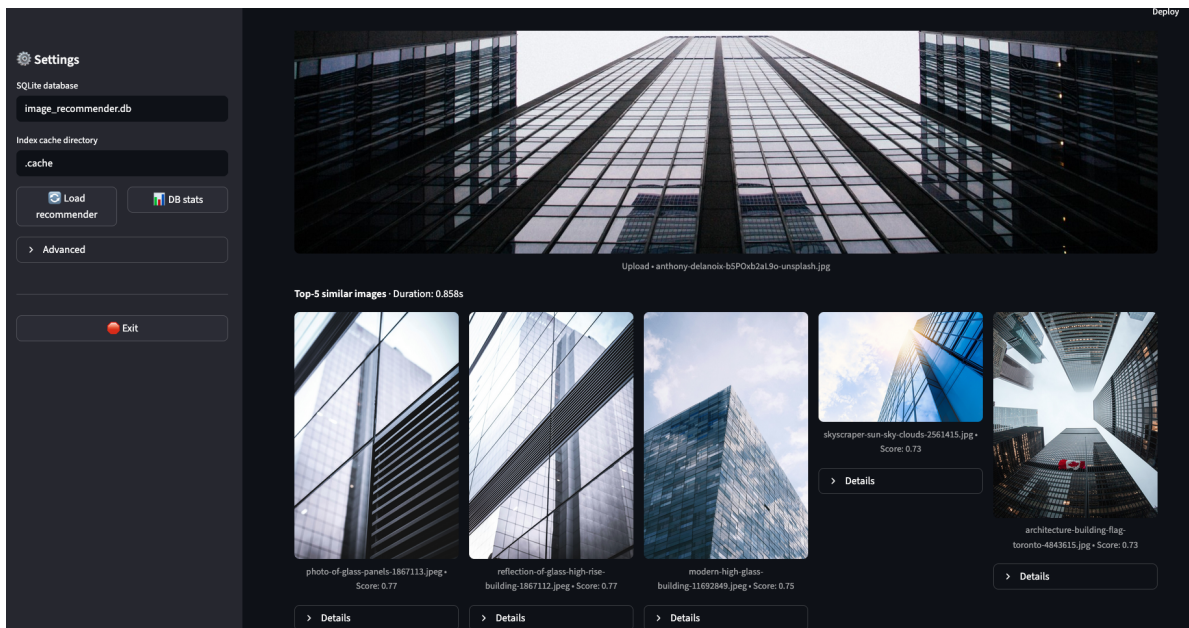


(a) Startansicht mit DB-Pfad und Cache-Verzeichnis (Sidebar) sowie Feature-Gewichten.

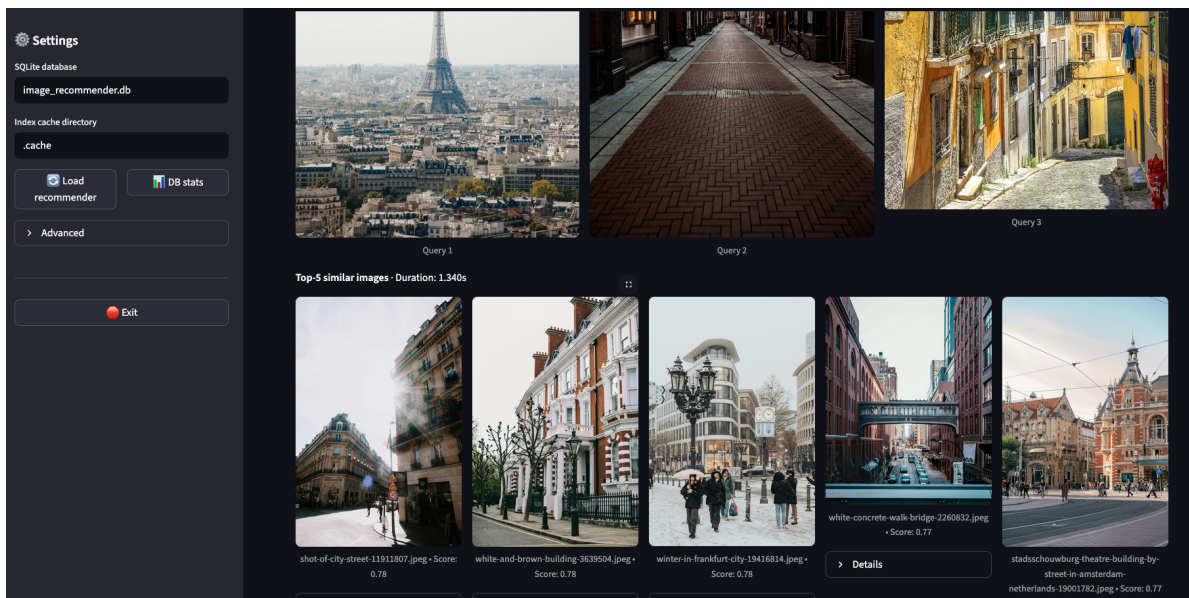


(b) Gewichte im *Custom*-Modus und Advanced-Parameter (*candidates*, *top-k*).

Abbildung 2: Streamlit-UI: Konfiguration & Schnellstart.



(a) Single-Query: Query-Vorschau (oben) und Top-5 Karten mit Score und Details (unten).



(b) Multi-Query: Drei Query-Bilder; Feature-Fusion (standardmäßig Mittelwert) und Ergebnisliste.

Abbildung 3: Interaktive Suche in der Streamlit-App.

Arbeitsweise. Beim Laden des Recommenders werden zunächst die *günstigen* Feature-Vektoren (Farbe/Stats) und, falls vorhanden, die *Deep-Embeddings* aus der DB gelesen. Für beide Teilmengen wird ein Index erstellt bzw. aus dem Cache geladen. Eine Abfrage besteht aus: (1) Feature-Berechnung der Query, (2) ANN/Linearsuche zur Kandidatenselektion, (3) Re-Ranking mit unserem Fusionsscore und (4) Anzeige mit Detailwerten (*color, embedding, custom*).

Troubleshooting.

- **Indizes bauen jedes Mal neu?** Stelle sicher, dass das Cache-Verzeichnis beschreibbar ist (.cache) und nicht geleert wird; DB-Pfad und Cache-Ordner dürfen zwischen Läufen nicht wechseln.
- **„No deep embeddings“-Hinweis:** Erst *Backfill* (§3.1) ausführen.

- **Exit-Button:** Schließt Browser-Tab (JS) und beendet den Streamlit-Prozess sauber.

4 Methoden zur Bildähnlichkeit

4.1 Farbfeatures (Histogramme, Stats, dominante Farben)

Wir extrahieren normalisierte **HSV**- und **BGR**-Histogramme (typisch 16–32 Bins/Kanal) sowie Helligkeit und Pro-Kanal-Mittelwerte/Standardabweichungen. Zusätzlich bestimmen wir k *dominante Farben* (KMeans in Lab/BGR, auf verkleinertem Bild). Die Farbsimilarität ergibt sich als gewichtete Summe von Kosinus-Ähnlichkeiten:

$$s_{\text{color}} = \alpha \cos(\mathbf{h}_1^{\text{HSV}}, \mathbf{h}_2^{\text{HSV}}) + \beta \cos(\mathbf{h}_1^{\text{BGR}}, \mathbf{h}_2^{\text{BGR}}) + \gamma \left(1 - \frac{|b_1 - b_2|}{255}\right),$$

mit $\alpha = \beta = 0.45$, $\gamma = 0.10$, und b als mittlerer Helligkeitswert.

4.2 Deep Embeddings (MobileNet V3, 128D)

Wir verwenden MobileNet V3 (Small oder Large) als Feature-Backbone, globales Pooling und eine fixe, deterministische Linearkombination ($R \in \mathbb{R}^{d \times 128}$) als Projektion auf 128D. Die Vektoren werden ℓ_2 -normiert. Die Ähnlichkeit ist (in $[0, 1]$ gepappt):

$$s_{\text{emb}} = \frac{\cos(\mathbf{z}_1, \mathbf{z}_2) + 1}{2}.$$

4.3 Custom-Maß: pHash + Varianz + Entropie

Zusätzlich verwenden wir einen **perceptual hash** (pHash, 64 Bit) mit DCT-Low-Frequenzen und bilden daraus die Ähnlichkeit:

$$s_{\text{phash}} = 1 - \frac{\text{Hamming}(h_1, h_2)}{64}.$$

Dazu kommen Varianzähnlichkeit der BGR-Std-Abweichungen und Entropie-Ähnlichkeit der Histogramme. Gesamt:

$$s_{\text{custom}} = 0.5 s_{\text{phash}} + 0.25 s_{\text{var}} + 0.25 s_{\text{ent}}.$$

4.4 Score-Fusion und Re-Ranking

Der finale Score ist eine gewichtete Fusion:

$$S = w_c s_{\text{color}} + w_e s_{\text{emb}} + w_u s_{\text{custom}}, \quad (w_c, w_e, w_u) = (0.4, 0.4, 0.2).$$

Wir wählen zunächst Kandidaten (Deep-Index bevorzugt) und re-ranken dann mit S .

4.5 Vergleich: Vor- und Nachteile

Maß	Vorteile	Nachteile
Farbe (HSV/BGR)	Schnell, robust bei Farbdominanzen, keine DL-Abhängigkeit	Sensitiv gegenüber Beleuchtung/Weißabgleich, weniger semantisch
Deep Embeddings	Semantische Ähnlichkeit, rotations-/skalierungsrobust (bedingt)	Bedarf DL-Runtime; Vorverarbeitung (Crop/Norm) beeinflusst Ergebnis
pHash/Custom	Erfasst grobe Struktur/Tonwerte, billig zu berechnen	Anfällig für starke Transformationen, nur globale Information

Tabelle 1: Vergleich unserer drei Similarity-Bausteine.

5 Pipeline: Ingestion und Backfill

5.1 Ingestion (schnelles Einlesen)

Ablauf pro Bild: *decode* → *optional resize* → *ColorAnalyzer* → *pHash* → DB-Write (Batches).
Optimierungen:

- **pyTurboJPEG** mit `-jpeg-reduce 8` (entspricht 1/8-Scaling beim Decoding) für JPEG.
- **ThreadPool** für I/O-Bound Arbeit; große `executemany`-Batches in einer Transaktion.
- **SQLite WAL** + PRAGMAs (`synchronous=NORMAL`, `mmap_size`, `cache_size`).
- **Downsizing** für Feature-Extraktion (`-resize ≈ 224–256` Pixel lange Kante).

5.2 Embedding-Backfill (batchweise, MPS/GPU)

Ablauf: Parallel dekodieren (Threads), dann *batchweise* ein Forward auf der GPU/MPS und Speicherung der 128D-Vektoren (BLOB). Wichtig ist ein großer `-batch` (z. B. 192) und `-jpeg-reduce 8`.

CLI-Beispiele (Ingest/Backfill)

```
1 # Ingest (NVMe/SSD, schneller Pfad mit TurboJPEG)
2 python ImageDatabase.py \
3   --ingest "/pfad/zue/bildern" \
4   --db image_recommender.db \
5   --pool thread --workers 64 \
6   --decode-mode auto --jpeg-reduce 8 \
7   --bins 16 --k 3 --resize 224 \
8   --batch-size 8000 --log-every 1000
9
10 # Backfill (Deep Embeddings, MPS/GPU)
11 python EmbedBackfill.py \
12   --db image_recommender.db \
13   --batch 192 \
14   --io-workers 16 \
15   --decode-mode auto \
16   --jpeg-reduce 8 \
17   --log-every 5
```

6 Suche und ANN

6.1 NumPy-Cosine vs. HNSW

Für kleinere Datenmengen ist ein **NumPy-Cosine-Index** (vorab normalisierte Vektoren + gespeicherte Normen) sehr schnell und simpel. Ab einem Schwellwert $N > 1000$ schalten wir auf **HNSW** (via `hnswlib`) um:

- Parameter: $M=16$, $ef_construction=200$, $ef_search=200$ (mindestens $\geq k$).
- Komplexität: Indexbau ist $O(N \log N)$ (heuristisch); Abfrage oft sublinear und sehr schnell in der Praxis.
- Fallback: Wenn `hnswlib` nicht installiert ist, nutzen wir automatisch NumPy-Cosine.

7 Performanceanalyse

7.1 Performanceanalyse und Optimierung

Während des Projekts zeigte sich, dass die größten Laufzeitkosten im **Ingest-Prozess** entstehen. Die durchschnittliche Geschwindigkeit begann bei ca. 40 Bildern/s, fiel jedoch bei längeren Läufen kontinuierlich auf ca. 23 Bilder/s ab. Der Grund liegt im **I/O-Bottleneck**: das ständige Lesen von Hunderttausenden JPEG-Dateien von einer externen HDD/SSD. Die eigentliche Feature-Berechnung (Resize, Color-Features, Perceptual Hash) benötigt dagegen nur wenige Millisekunden pro Bild.

Auch beim **Backfill der Deep Embeddings** kam es zu deutlichen Laufzeiten. Mit einem MPS-fähigen Apple-GPU-Backend erreichten wir ca. 5–15 Bilder/s. Zeitweise aber auch bis zu 160 Bilder/s. Dies reicht aus, um mittelgroße Datensätze in Stunden zu verarbeiten, wird jedoch bei hunderttausenden Bildern zu einem Flaschenhals.

Wir testeten verschiedene Parameter (`batch size`, `io-workers`) und stellten fest:

- `batch size > 512` brachte keinen Vorteil, sondern höhere Latenz.
- `io-workers > 16` führte zu Diminishing Returns, da die I/O-Quelle limitiert.
- Gleichzeitiges Ausführen von Ingest und Backfill reduzierte die Ingest-Rate deutlich (von ~ 40 auf unter 25 imgs/s). Daher wurde entschieden, die Prozesse sequenziell statt parallel zu fahren.

Versuchsaufbau (Hardware & Datensatz)

- **Rechner:** [Apple Silicon Mx / n Cores], [RAM xx GB], [macOS xx], Python 3.9.
- **Datensatz:** $\sim 342,890$ JPEGs (Ingest), Zielkorpus $\sim 500k$; SQLite-DB mit drei Tabellen.
- **Konfiguration:** `-workers 64`, `-jpeg-reduce 8`, `-resize 224`, `-batch 192`, `-io-workers 16`, HNSW ($M=16$, $efC=200$, $efS=200$).

Szenario	Top-5 Dauer	Bemerkung
Warmstart (Deep+Cache)	0.30–0.90 s	HNSW aus Cache geladen
Kaltstart (Index-Build)	~ 35 s (einmalig)	cheap+deep Indexaufbau
Multi-Query (3 Bilder)	1.0–1.4 s	Fusion, Re-Ranking $k = 5$

Tabelle 2: Beobachtete Query-Latenzen (repräsentativ; je nach Hardware leicht variierend).

8 Machbarkeitsanalyse

Insgesamt lässt sich unser System auf einige hunderttausend Bilder skalieren. Die Kombination aus **SQLite-Datenbank** + **ANN-Index** erlaubt schnelle Abfragen: ein typischer Top-5 Query benötigt < 0.5 Sekunden bei aktiviertem ANN. Ohne ANN steigt die Abfragezeit linear mit der Datenbankgröße, was ab etwa 10^5 Bildern nicht mehr praktikabel ist.

Die Hauptlimitation ist das **Einfügen neuer Daten** (Ingest + Backfill), welches zeitintensiv bleibt. Für ein kontinuierliches Echtzeit-System wäre eine Streaming-Pipeline notwendig. Als Optimierungsansätze diskutieren wir:

- Einsatz einer schnelleren GPU für Embeddings.
- Speicherung der Bilder auf NVMe-SSDs oder im RAM-Cache, um I/O zu entlasten.
- Nutzung eines spezialisierteren ANN-Frameworks (z.B. Faiss, hnswlib).

Stufe	Durchsatz [imgs/s]	Decode [ms/img]	Embed [ms/img]
Ingest (HDD/USB)	7–22	≈ 1500	–
Backfill (MPS)	35–45	≈ 19	$\approx 3\text{--}6$

Tabelle 3: Ausgewählte gemessene Werte (repräsentativ für unseren Teststand).

Messwerte (CLI-Logs)

```
INFO:ImageDatabase:... 317779/353127 | 27.2 imgs/s | elapsed 03:14:52 | ETA 00:21:40 (~23:51:53) | avg ms/img: io 2343.8 | resize 1.6 | color 6.1 | phash 0.13 | total 2354.5
INFO:ImageDatabase:... 319778/353127 | 27.3 imgs/s | elapsed 03:15:09 | ETA 00:20:21 (~23:50:51) | avg ms/img: io 2332.4 | resize 1.6 | color 6.1 | phash 0.13 | total 2343.1
INFO:ImageDatabase:... 321778/353127 | 27.4 imgs/s | elapsed 03:15:26 | ETA 00:19:02 (~23:49:40) | avg ms/img: io 2321.3 | resize 1.6 | color 6.1 | phash 0.13 | total 2332.0
INFO:ImageDatabase:... 323776/353127 | 27.6 imgs/s | elapsed 03:15:43 | ETA 00:17:44 (~23:48:48) | avg ms/img: io 2310.3 | resize 1.5 | color 6.1 | phash 0.13 | total 2320.9
INFO:ImageDatabase:... 325772/353127 | 27.7 imgs/s | elapsed 03:16:02 | ETA 00:16:27 (~23:47:49) | avg ms/img: io 2299.7 | resize 1.5 | color 6.1 | phash 0.13 | total 2310.3
INFO:ImageDatabase:... 327772/353127 | 27.8 imgs/s | elapsed 03:16:19 | ETA 00:15:11 (~23:46:50) | avg ms/img: io 2289.0 | resize 1.5 | color 6.1 | phash 0.13 | total 2299.5
INFO:ImageDatabase:... 329770/353127 | 28.0 imgs/s | elapsed 03:16:36 | ETA 00:13:55 (~23:45:51) | avg ms/img: io 2278.4 | resize 1.5 | color 6.1 | phash 0.13 | total 2288.9
INFO:ImageDatabase:... 331770/353127 | 28.1 imgs/s | elapsed 03:16:55 | ETA 00:12:40 (~23:44:56) | avg ms/img: io 2268.4 | resize 1.5 | color 6.0 | phash 0.13 | total 2278.8
INFO:ImageDatabase:... 333770/353127 | 28.2 imgs/s | elapsed 03:17:13 | ETA 00:11:26 (~23:43:59) | avg ms/img: io 2258.2 | resize 1.5 | color 6.0 | phash 0.13 | total 2268.6
INFO:ImageDatabase:... 335768/353127 | 28.3 imgs/s | elapsed 03:17:31 | ETA 00:10:12 (~23:43:04) | avg ms/img: io 2248.2 | resize 1.5 | color 6.0 | phash 0.13 | total 2258.6
INFO:ImageDatabase:... 337769/353127 | 28.5 imgs/s | elapsed 03:17:48 | ETA 00:08:59 (~23:42:06) | avg ms/img: io 2238.1 | resize 1.5 | color 6.0 | phash 0.13 | total 2248.4
INFO:ImageDatabase:... 339766/353127 | 28.6 imgs/s | elapsed 03:18:07 | ETA 00:07:47 (~23:41:15) | avg ms/img: io 2228.5 | resize 1.5 | color 6.0 | phash 0.13 | total 2238.8
INFO:ImageDatabase:... 341764/353127 | 28.7 imgs/s | elapsed 03:18:26 | ETA 00:06:35 (~23:40:22) | avg ms/img: io 2218.9 | resize 1.5 | color 6.0 | phash 0.13 | total 2229.2
INFO:ImageDatabase:Bulk-Ingest fertig: added=342898, errors=18237 | avg ms/img: io 2213.3 | resize 1.5 | color 6.0 | phash 0.13 | total 2223.6 | throughput 28.8 imgs/s | elapsed 11915.9s | workers=64
pool-thread decode=turbo jpeg reduce=8 | decoders {turbo: 342898, 'imcode': 0, 'opencv': 0, 'skip': 0} | formats {'jpeg': 342898, 'other': 0}
```

(a) **Ingestion:** Durchsatz steigt gegen Ende auf $\approx 28\text{--}29$ imgs/s. Rechts unten sind die zerlegten Mittelwerte pro Bild zu sehen: *io* (I/O) $\approx 2,2$ s/img, *resize* $\approx 1,5$ ms/img, *color* ≈ 6 ms/img, *phash* $\approx 0,13$ ms/img. Die Dominanz von I/O bestätigt den Flaschenhals beim Dateizugriff.

```
... 122881/129458 | 49.8 imgs/s (global), 0.0 (chunk) | elapsed 00:41:08 | ETA 00:02:12 (~03:33:00)
... 124881/129458 | 50.5 imgs/s (global), 463.6 (chunk) | elapsed 00:41:13 | ETA 00:01:30 (~03:32:23)
... 126881/129458 | 51.2 imgs/s (global), 446.9 (chunk) | elapsed 00:41:17 | ETA 00:00:50 (~03:31:47)
... 128881/129458 | 51.9 imgs/s (global), 459.8 (chunk) | elapsed 00:41:21 | ETA 00:00:11 (~03:31:13)
... 129458/129458 | 52.1 imgs/s (global), 438.2 (chunk) | elapsed 00:41:23 | ETA 00:00:00 (~03:31:03)
[DONE] 129458/129458 in 00:41:23 (52.1 imgs/s)
```

(b) **Backfill (Deep Embeddings):** 129,458 Bilder in 00:41:23, mittlerer Durchsatz $\approx 52,1$ imgs/s. Die Last liegt vor allem beim GPU/MPS-Forward und Batch-Dekodieren.

Abbildung 4: CLI-Screenshots der Abschluss-Metriken für Ingest und Backfill.

Interpretation.

- **Ingest:** Der sichtbare Durchsatz von $\approx 28\text{--}29$ imgs/s ist für externe Laufwerke plausibel. Die zerlegten Zeiten (*io* $\gg$ *resize/color/phash*) zeigen klar: **I/O dominiert**. Optimierungen zielen daher auf Speichermedium (NVMe), paralleles Lesen, und möglichst billiges Decoding (TurboJPEG mit `-jpeg-reduce`).

- **Backfill:** ≈ 52 imgs/s mit MPS/GPU ist solide; die Rate skaliert mit Batch-Größe und I/O-Vorladen. Größere Batches nutzen die GPU besser, zu groß erhöht jedoch die Latenz.
- **Implikation für Skalierung:** Sobald die Embeddings vorliegen und der HNSW-Index aus dem Cache geladen wird, sind Online-Abfragen < 0.5 s für Top-5 realistisch (siehe Such-Logs).

9 Big-Data-Analyse: 2D/3D-Visualisierung

Wir projizieren die 128D-Embeddings in 2D oder 3D, sodass ähnliche Bilder nahe beieinander liegen.

10 Visualisierung der Embeddings

Zur qualitativen Analyse der 128-dimensionalen Mobilenet-Embeddings projizieren wir zufällig gezogene Teilmengen mit **UMAP** und **t-SNE** in 2D/3D. Für Performance wurden $N=60\,000$ Punkte für UMAP und $N=20\,000$ für t-SNE verwendet. Wir normieren die Vektoren auf L2-Norm (cosine $\approx$ euclid auf normierten Vektoren) und fixieren die Zufallskeime, um Reproduzierbarkeit zu gewährleisten.

10.1 Overlay: Helligkeit (Brightness)

Abb. 5 zeigt UMAP/t-SNE-Projektionen, eingefärbt nach der durchschnittlichen Bildhelligkeit (Feature `color_stats.brightness`).

Beobachtungen. UMAP bildet zwei bis mehrere *Makro-Cluster*, die in 3D noch klarer separiert sind. Das Brightness-Overlay verteilt sich innerhalb dieser Cluster relativ gleichmäßig; d. h. Helligkeit ist *nicht* der dominante Treiber der Struktur. t-SNE erzeugt kompaktere „Wolken“, zeigt aber ein ähnliches Bild: Helligkeitsgradienten verlaufen innerhalb von Clustern, ohne diese allein zu erklären.

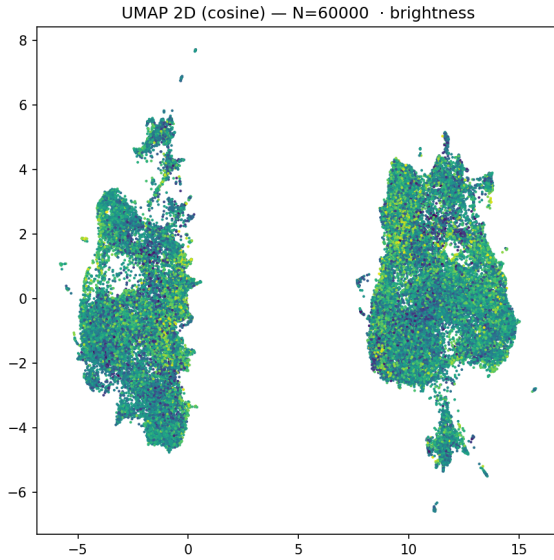
10.2 Overlay: Farbton (Hue)

Abb. 6 wiederholt die Visualisierung, diesmal eingefärbt nach dem *HSV-Farbton* (Hue) je Bild. Der Farbton nimmt zyklische Werte an (Rot $\rightarrow$ Gelb $\rightarrow$ Grün $\rightarrow$ Cyan $\rightarrow$ Blau $\rightarrow$ Magenta $\rightarrow$ Rot), weshalb sich farbige Bänder und Inseln ergeben können.

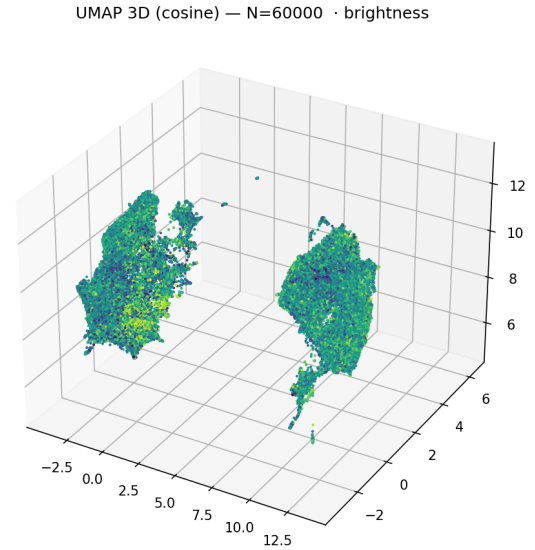
Beobachtungen. Im Hue-Overlay sind Farbtöne über viele Regionen gemischt, was darauf hindeutet, dass die Deep-Embeddings semantische/strukturelle Merkmale (Form, Textur, Motiv) stärker kodieren als reine Farbe. Gleichzeitig zeigen sich lokal *Hue-kohärente Bereiche* (z. B. Ansammlungen ähnlich gefärbter Objekte), was für die spätere Farbkomponente im Re-Ranking nützlich ist. Dass UMAP zwei dominante Inseln liefert, während t-SNE eher eine flockige Partitionierung zeigt, ist erwartbar: UMAP bewahrt globale Nachbarschaften besser, t-SNE betont lokale Nachbarschaften (Perplexity-Skala). In 3D sind Übergänge zwischen Motivklassen sichtbarer; 2D und 3D ergänzen sich.

10.3 Einordnung für die Suche

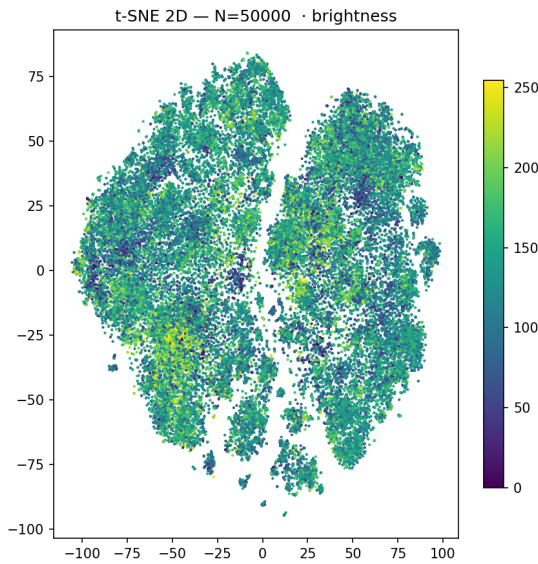
Die Visualisierung bestätigt, dass (i) Farbe allein die Struktur nicht bestimmt und (ii) Deep-Features sinnvolle semantische Nachbarschaften liefern. Das rechtfertigt unsere *Hybridwertung* im Recommender (Gewichte für *embedding/color/custom*). Praktisch heißt das: nahe Punkte in



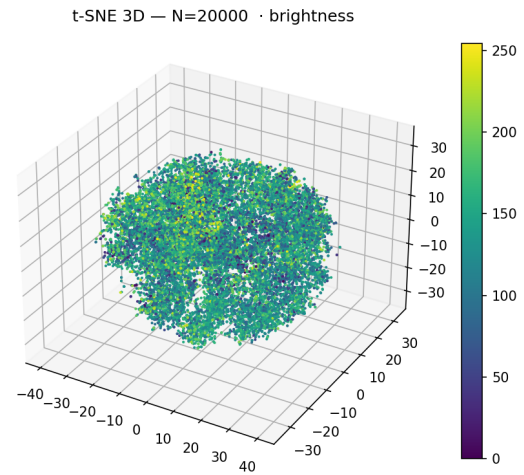
(a) UMAP 2D (cosine), $N = 60\,000$



(b) UMAP 3D (cosine), $N = 60\,000$



(c) t-SNE 2D, $N = 20\,000$



(d) t-SNE 3D, $N = 20\,000$

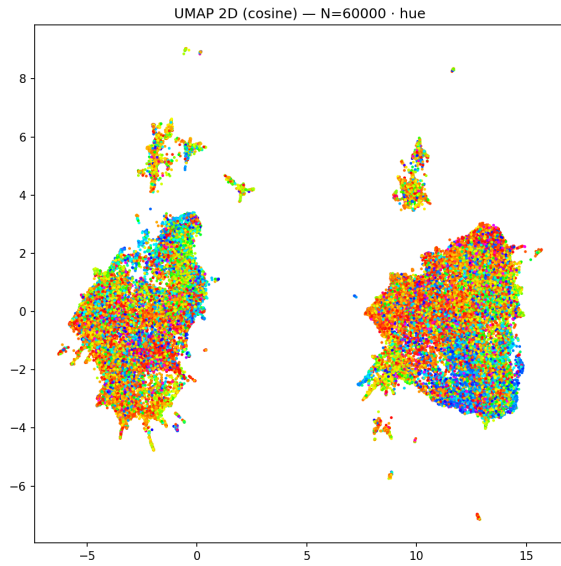
Abbildung 5: Embedding-Projektionen, eingefärbt nach *Brightness*. Helle Punkte entsprechen hohen Grauwerten.

UMAP/t-SNE führen häufig zu qualitativ ähnlichen Treffern; das Hue/Brightness-Overlay hilft, Farb-Bias zu erkennen und ggf. die Gewichte anzupassen.

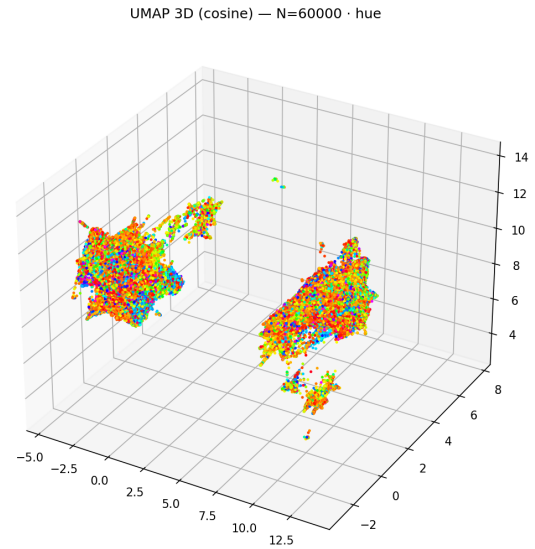
Parameter & Reproduzierbarkeit. UMAP: `metric=cosine, n_neighbors=15, min_dist=0.1`; t-SNE: `init=pca`, adaptive Perplexity ($\leq N/3$); beide mit fixem `random_state`. Bei größeren N steigt die Laufzeit quadratisch (t-SNE) bzw. nahezu linear (UMAP); deshalb unterscheiden sich die Stichprobengrößen.

Diskussion der Ergebnisse

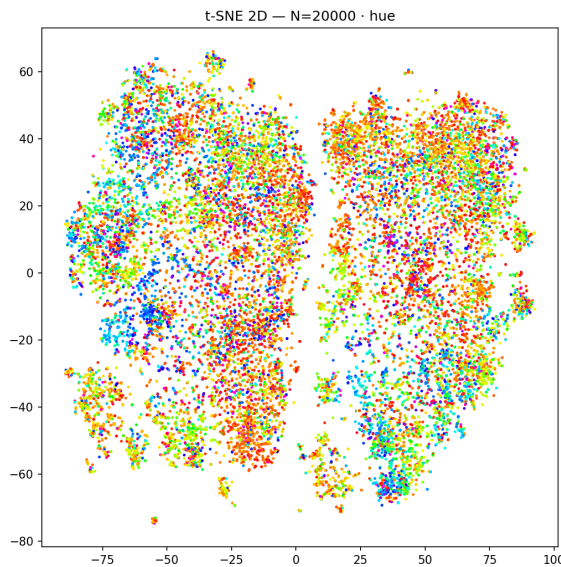
- **Lokale Struktur (t-SNE).** Es entstehen mehrere kompakte Inseln – das Embedding bildet konsistente Nachbarschaften ab. Da die Brightness innerhalb der Cluster gemischt ist,



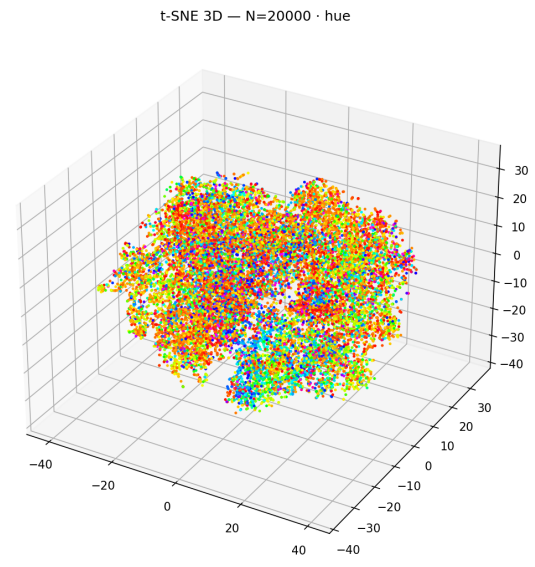
(a) UMAP 2D (cosine), $N = 60\,000$



(b) UMAP 3D (cosine), $N = 60\,000$



(c) t-SNE 2D, $N = 20\,000$



(d) t-SNE 3D, $N = 20\,000$

Abbildung 6: Embedding-Projektionen, eingefärbt nach *Hue* (HSV-Farbtone).

resultiert die Gruppierung nicht nur aus Helligkeit, sondern aus semantischen Eigenschaften (Texturen, Motiv, Farbfamilien).

- **Globalstruktur (UMAP 2D).** UMAP erhält mehr globale Geometrie und zeigt zwei größere Bereiche mit Subclustern. Die Trennung korreliert nicht trivial mit Brightness; Treiber sind eher Farbpaletten/Hue, Motivklassen oder Bildgeometrie.
- **Tiefe Sicht (UMAP 3D).** In 3D werden Übergänge/Trennungen klarer – die beobachtete Struktur ist kein 2D-Artefakt. Das stützt die Qualität des 128-D-Raums für k-NN.

TMAP (Versuch). Wir haben TMAP prototypisch ausprobiert, stießen jedoch auf eine API-Differenz (`module 'tmap' has no attribute 'LSHForest'`). Aufgrund der lokalen Paketversion konnten wir den MinHash/LSH-Flow nicht reproduzieren und haben TMAP nicht in die finale Evaluation aufgenommen. (UMAP/t-SNE decken die visuelle Analyse ab.)

Fazit Die Visualisierungen bestätigen, dass das tiefe Embedding sinnvolle Nachbarschaften lernt und sich nicht auf simple Helligkeitsunterschiede reduziert. Dies erklärt die guten Retrieval-Ergebnisse und rechtfertigt die Wahl der *cosine*-Metrik sowie die Re-Ranking-Strategie (Farb-/Embedding-/Custom-Score).

11 Tests und Qualitätssicherung

Ziel der Qualitätssicherung ist reproduzierbare Korrektheit, saubere Codebasis (PEP 8) und einfache Ausführbarkeit auf macOS, Linux und Windows. Wir kombinieren *Linting & Formatting* (Ruff) mit *Unit-Tests* (pytest).

11.1 Linting & Formatting (Ruff)

Für statische Analyse und Auto-Fixes nutzen wir **Ruff**. Damit wurden u. a. moderne Typannotationen (PEP 585/604), unbenutzte Variablen und Import-Probleme bereinigt und ein einheitliches Format erzwungen.

Status zur Abgabe: Alle Ruff-Checks bestehen, Format ist einheitlich.

11.2 Unit-Tests (pytest)

Wir verwenden **pytest** mit einer schlanken Testsuite unter `tests/`. Abgedeckt sind:

- **Similarity-Funktionen:** Farb-, Embedding- und Custom-Scores. Tests prüfen Identität, Helligkeits-Penalty und robuste Kosinus-Ähnlichkeit (deterministisch via Seeds/synthetische Vektoren).
- **Feature-Extraktion:** Texturfeatures (Gradient-Histogramm, Laplacian-Varianz, Entropie) und *pHash* (Gleichheit/Robustheit).
- **Datenbank:** Schema-Smoke, Einfüge-/Lese-Roundtrip und Basismetriken (`get_database_stats`).
- **ANN/Index:** Aufbau eines kleinen Index (Fallback auf NumPy-Cosine oder HNSW, falls verfügbar) und `knn`-Abfrage.
- **CLI-Hilfsfunktionen:** Dateiauffistung mit `iter_images`.
- Für schnelle End-to-End-Checks ohne großen Datensatz nutzen wir **Smoke-Tests** (`@pytest.mark.smoke`). Die Tests erzeugen temporäre Mini-Bilder ($\approx 10 \times 10$ px), bootstrappen eine minimale SQLite-DB (Schema + zwei Zeilen) und rufen die CLI über `subprocess` auf. So prüfen wir Ende-zu-Ende: *Datenbank* $\rightarrow$ *Recommender* $\rightarrow$ *CLI-Ausgabe*.

Ergebnis: 11 Tests, alle bestanden (`pytest -q`).

12 Schwierigkeiten und Lessons Learned

- **I/O-Bottleneck:** Externe/rotierende Datenträger limitieren Ingest massiv. NVMe/SSD ist der Schlüssel.
- **Decoderwahl:** OpenCV-Reduced-Decode ist hilfreich, aber **pyTurboJPEG** mit 1/8-Scaling war deutlich schneller und CPU-schonender.
- **Stabilität:** Dekodierfehler einzelner Dateien dürfen den Lauf nicht abbrechen; robustes Logging und Skip-Strategie sind wichtig.

- **ANN-Fallback:** `hnswlib` ist nicht überall installiert; unser NumPy-Cosine-Fallback hält die Software dennoch nutzbar.
- **Reproduzierbarkeit:** Deterministische Seeds (KMeans, Projektion), feste Preprocess-Steps und klare CLI-Parameter vermeiden „Heisenbugs“.

13 Fazit

Wir haben eine performante, modulare Bildähnlichkeitssuche konzipiert und implementiert. Die Kombination aus **Farbprofilen**, **Deep Embeddings** und **pHash-basierten** Merkmalen liefert robuste Resultate, während **TurboJPEG**, **Batching** und **WAL** die Laufzeit deutlich verbessern. Mit einem HNSW-Index und vorab berechneten Embeddings sind Top-5-Anfragen in der Regel in deutlich unter einer Sekunde möglich. Für sehr große Datenmengen bleibt die Hardware (v. a. I/O) der größte Hebel; die Software-Architektur ist bereit, diese Ressourcen effektiv auszunutzen.

A Anhang A: Datenbankschema (SQL)

```

1 CREATE TABLE IF NOT EXISTS images (
2     image_id      TEXT PRIMARY KEY,
3     filename      TEXT NOT NULL,
4     filepath      TEXT NOT NULL,
5     file_size     INTEGER,
6     width         INTEGER,
7     height        INTEGER,
8     channels       INTEGER,
9     file_hash     TEXT UNIQUE,
10    photographer   TEXT,
11    created_date   TEXT,
12    added_to_db    TEXT,
13    UNIQUE(filepath)
14 );
15 CREATE TABLE IF NOT EXISTS color_features (
16     image_id      TEXT PRIMARY KEY,
17     dominant_colors TEXT,
18     hsv_histogram  TEXT,
19     bgr_histogram  TEXT,
20     color_stats    TEXT,
21     FOREIGN KEY (image_id) REFERENCES images(image_id)
22 );
23 CREATE TABLE IF NOT EXISTS advanced_features (
24     image_id      TEXT PRIMARY KEY,
25     texture_features TEXT,
26     deep_embeddings BLOB,
27     custom_features TEXT,
28     deep_dim       INTEGER,
29     FOREIGN KEY (image_id) REFERENCES images(image_id)
30 );
31 CREATE INDEX IF NOT EXISTS idx_images_path ON images(filepath);
32 CREATE INDEX IF NOT EXISTS idx_images_hash ON images(file_hash);

```